Lab 08 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — Drie images voor drie omgevingen

Antwoord. De image die in acceptatie getest is (`-staging`) is **niet** die welke in productie uitgerold wordt (`-prod`): twee builds, twee inhouden, en de productieconfiguratie is nergens beproefd vóór de productie. Er had **één enkele image** `api:1.4.2` moeten zijn, één keer gebouwd, met de configuratie ingespoten bij de uitrol (`--env-file`, gemount bestand, secrets). Je verliest: de garantie "wat getest is, is wat draait", de traceerbaarheid via digest, de mogelijkheid om dezelfde image te promoveren; en je wint drie builds, drie scans, drie keer opslag.

Waarom. Een image is een onveranderlijk artefact, geïdentificeerd door zijn digest; die digest promoveer je. Zodra de configuratie erin zit, verandert het artefact met de omgeving.

Nuance. Een `ENV` met een **standaardwaarde** in de image (`APP_PROFILE=default`, `SERVER_PORT=8080`) is legitiem: die beschrijft de image, niet een omgeving.

Voorbeeld.

```
podman run -d --name api-staging --env-file staging.env api:1.4.2
podman run -d --name api-prod    --env-file prod.env    api:1.4.2     # zelfde image, zelfde digest
```

Vraag 2 — Voorrang van de bronnen

Antwoord. `staging`, `prod`, `prod`, `shell`. De volgorde van de opties speelt geen rol omdat Podman de bronnen niet "in de volgorde van de regel" toepast maar per **categorie**: eerst de `ENV` van de image, dan de bestanden van `--env-file`, dan de `-e`-opties, waarbij de laatste de eerste overschrijven. `-e APP_PROFILE` zonder waarde kopieert de variabele van de shell van de host.

Waarom. Het is een voorrangsregel, geen reeks: de waarde die het "dichtst" bij het commando staat, wint (commandoregel > bestand > image).

Nuance. In een `--env-file` wint de **laatste** regel voor eenzelfde sleutel; en Docker hanteert dezelfde regel `-e` > `--env-file`.

Voorbeeld.

```
podman run --rm -e APP_PROFILE=prod --env-file api.env api:env env | grep APP_PROFILE   # prod
```

Vraag 3 — Poort 9090 en `-p 18080:8080`

Antwoord. `SERVER_PORT=9090` liet de applicatie luisteren op **9090**; `-p 18080:8080` geeft door naar 8080, waar niemand meer luistert: verbinding geweigerd in de container, `curl` zonder antwoord. `EXPOSE 8080` verandert daar niets aan: het beschrijft de image, niet de echte configuratie van het proces. Je moet ze op elkaar afstemmen: `-p 18080:9090`, of de poort niet veranderen.

Waarom. Drie onafhankelijke lagen: de applicatie luistert waar haar configuratie het zegt, `-p` geeft door waar jij het zegt, `EXPOSE` documenteert. Geen van de drie controleert de twee andere.

Nuance. In een user-defined netwerk (lab 07) zouden de andere containers dan ook `api:9090` moeten aanroepen. Een poort via een variabele veranderen heeft een grotere reikwijdte dan het lijkt.

Voorbeeld.

```
podman run -d --name api2 -p 18080:9090 -e SERVER_PORT=9090 api:1.0 && sleep 2 && curl -s localhost:18080/actuator/health
```

Vraag 4 — Wachtwoord als argument

Antwoord. Van ernstig naar minder ernstig: (1) **de logs** — de applicatie drukt haar argumenten af (`Arguments recus : --db-password=Secr3t`), en logs worden gekopieerd, gecentraliseerd, bewaard; (2) **de `ps` van de host** — elke gebruiker van de machine leest `ps -eo args`; (3) **`podman inspect`** (`Config.Cmd`) — iedereen met toegang tot de engine; (4) **`podman top api args`** — idem. Daar komt de shellgeschiedenis nog bij (`~/.bash_history`).

Waarom. De commandoregel van een proces is publieke informatie van het systeem (`/proc/<pid>/cmdline`), en het commando van een container wordt in zijn configuratie bewaard.

Nuance. Ook zonder dat de applicatie iets afdrukt, volstaat `ps`. Een argument is nooit een kanaal voor een secret, in of buiten een container.

Voorbeeld.

```
podman logs api | head -1 ; podman inspect api --format '{{json .Config.Cmd}}' ; ps -eo args | grep -c 'db-password=Secr3t'
```

Vraag 5 — Wachtwoord als variabele

Antwoord. Ja, dat is beter: het verdwijnt uit de argumentlogs, uit `ps` en uit `Config.Cmd`. Het blijft zichtbaar in `podman inspect` (`Config.Env`), `podman exec api env`, `/proc/1/environ` in de container, en in elk foutrapport dat de omgeving dumpt. Met `POSTGRES_PASSWORD_FILE=/run/secrets/db_password` bevat `Config.Env` alleen nog dat **pad**: de waarde is alleen leesbaar voor een proces in de container dat het bestand opent.

Waarom. De omgeving is metadata van de container, bewaard door de engine; een gemount bestand is dat niet.

Nuance. `podman exec db cat /run/secrets/db_password` blijft mogelijk voor wie toegang heeft tot de engine. De winst is dat de waarde uit de **metadata** en de exports verdwijnt (`inspect`, Compose, tickets), niet dat ze onbereikbaar wordt voor de beheerder.

Voorbeeld.

```
podman inspect db --format '{{range .Config.Env}}{{println .}}{{end}}' | grep POSTGRES   # POSTGRES_PASSWORD_FILE=/run/secrets/db_password
```

Vraag 6 — Spring Boot en de omgeving

Antwoord. *Relaxed binding* zet een variabelenaam om in een propertynaam: hoofdletters → kleine letters, `_` → `.` (`SPRING_DATASOURCE_URL` → `spring.datasource.url`). `SPRING_PROFILES_ACTIVE=prod` activeert het profiel `prod`: Spring Boot laadt bovendien

`application-prod.yaml` en de beans met `@Profile("prod")`. Bij een conflict wint de **omgevingsvariabele** van `/config/application.yaml` (extern bestand, buiten de JAR), dat wint van `application.yaml` in de JAR.

Waarom. Spring Boot ordent zijn `PropertySources` van het meest externe naar het meest interne, precies om inspuiten zonder rebuild mogelijk te maken.

Nuance. De argumenten `--spring.datasource.url=…` gaan vóór de variabelen — maar dat zijn argumenten, dus zichtbaar (vraag 4). En `spring.config.import=optional:configtree:/run/secrets/` maakt van elk bestand in `/run/secrets/` een property.

Voorbeeld.

```
podman run -d --name api -e SPRING_PROFILES_ACTIVE=prod -e
SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/app -v
./application.yaml:/config/application.yaml:ro echte-api:1.0
```

Vraag 7 — Betwistbare Dockerfile

Antwoord. `ENV SPRING_PROFILES_ACTIVE=prod`: legt een omgeving vast in de image — hoort via `-e` bij de uitrol. `ENV SPRING_DATASOURCE_PASSWORD=Pr0d!`: een secret in de image, leesbaar via `image inspect` en `history` — hoort via een secret (`_FILE` / `configtree`). `COPY .env /app/.env`: een lokaal configuratiebestand, waarschijnlijk met secrets, in een laag — hoort in `.dockerignore`, en wordt bij `run` geleverd via `--env-file` of een mount. Gecorrigeerd:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/api.jar /app/api.jar
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/api.jar"]
```

Waarom. De image mag alleen bevatten wat overal waar is: de code en zijn standaardwaarden. Al de rest wordt ingespoten.

Nuance. Een `ENV SPRING_PROFILES_ACTIVE=default` of `ENV JAVA_TOOL_OPTIONS=-XX:MaxRAMPercentage=75` zou aanvaardbaar blijven: standaardwaarden, geen omgevingen.

Voorbeeld.

```
podman image inspect api --format '{{range .Config.Env}}{{println .}}{{end}}' | grep -i -E
'password|profile'   # moet leeg zijn
```

Vraag 8 — `podman secret` in de praktijk

Antwoord. `Config.Env` bevat `POSTGRES_PASSWORD_FILE=/run/secrets/db_password` en verder niets over het wachtwoord; `ls -l /run/secrets/` toont `-r--r--r-- 1 root root 6 … db_password`; `podman secret inspect db_password` geeft het ID, de datums, de driver `file` en zijn pad, **niet de waarde**. `--showsecret` toont ze wel: het bewijs dat de opslag een opbergplaats is die je gebruiker kan lezen, geen versleuteling.

Waarom. Het secret wordt bij het starten als bestand naar de container gekopieerd; de engine zet het noch in de omgeving, noch in de configuratie van de container.

Nuance. Het bestand is van `root` en leesbaar voor iedereen in de container (`444`): een niet-rootproces leest het. `--secret db_password,mode=0400,uid=70` beperkt de rechten.

Voorbeeld.

```
podman secret inspect --showsecret db_password --format '{{.SecretData}}'   # S3cr3t
podman inspect db --format '{{json .Config.Env}}' | grep -c 'S3cr3t'          # 0
```

Vraag 9 — "Dan kun je even goed `-e` gebruiken"

Antwoord. Ten opzichte van `-e` haalt `podman secret` de waarde weg uit `podman inspect`, uit de omgeving van het proces (in bestandsmodus), uit Compose-bestanden en uit de commando's in de shellgeschiedenis; het maakt ze **één keer** aan en verwijst er overal bij naam naar; en het mount ze met instelbare rechten. Wat het niet brengt: versleuteling in rust, toegangscontrole, een logboek van wie het las, rotatie. Dat is precies wat een secretsbeheerder toevoegt: wie wat wanneer gelezen heeft, versleuteling met een hoofdsleutel, automatische vernieuwing en kortlevende tokens.

Waarom. `podman secret` lost het probleem van de **toevallige blootstelling** op; een beheerder lost dat van de **governance** op.

Nuance. Op een ontwikkelmachine is `podman secret` ruim voldoende. De driver `pass` (GPG) bestaat om lokaal te versleutelen.

Voorbeeld.

```
podman secret ls ; podman secret inspect db_password --format '{{.Spec.Driver.Name}}'   # file
```

Vraag 10 — De build slaagt zonder secret

Antwoord. De tweede build vond al zijn instructies **in de cache**, ook de `RUN --mount=type=secret`, die dus niet opnieuw uitgevoerd werd — en een `RUN` die niet uitgevoerd wordt, vraagt geen secret. Om hem te zien falen: `podman build --no-cache`, of een vroegere instructie wijzigen. Gevolg: een verlopen token wordt niet opgemerkt zolang de cache standhoudt; op een CI-agent met blijvende cache "werkt" de build tot de dag dat een afhankelijkheid verandert.

Waarom. De cache wordt geïndexeerd op de instructie en de context, niet op de aangeleverde secrets: een secret wel of niet aanleveren maakt hem niet ongeldig.

Nuance. Dat is ook een bescherming: omdat het secret niet in de cachesleutel zit, is zijn waarde niet uit de cache af te leiden.

Voorbeeld.

```
podman build --no-cache -t app .   # Error: … secret required but no secret with id "token" found
```

Vraag 11 — `ARG` tegenover `--mount=type=secret`

Antwoord. Met `ARG NPM_TOKEN` zit het token in de metadata van de laag: `podman history --no-trunc` toont `|1 NPM_TOKEN=… /bin/sh -c npm ci`. Met de secret mount zit het nergens: `history` toont de regel `--mount=type=secret,id=npm_token npm ci`, zonder waarde, en `/run/secrets` bestaat niet in de image. De multi-stage beschermt de `ARG` **gedeeltelijk**: staat de `ARG` in een weggegooide stage, dan heeft de eindimage er geen geschiedenis van. Maar een `ARG` in de laatste stage blijft zichtbaar, en de image van de build-stage (`--target`) bewaart alles.

Waarom. `ARG` wordt gesubstitueerd in het opgeslagen commando van de laag; een mount wordt niet opgeslagen.

Nuance. Buildah en BuildKit behandelen beide op dezelfde manier. Het enige verschil: Buildah negeert `# syntax=` en heeft het niet nodig.

Voorbeeld.

```
podman history --no-trunc app --format '{{.CreatedBy}}' | grep -i token
```

Vraag 12 — `.env` in Git

Antwoord. (1) **Git vergeet niet**: zelfs na verwijdering in de volgende commit blijft het bestand in de geschiedenis, in alle clones en alle forks. (2) **Ontwikkeling staat niet los van de rest**: dev-wachtwoorden geven vaak toegang tot gedeelde diensten (testdatabase, registry, externe API in sandboxmodus), en lekscanners (Gitleaks, GitHub secret scanning) zullen het melden. (3) **De gewoonte**: de reflex "het is toch maar dev" eindigt met het committen van een acceptatiewaarde. Praktijk: `.env` in `.gitignore` en `.dockerignore`, een gecommit `.env.example` met de sleutels en lege waarden, en voor CI secrets die de tool inspuit.

Waarom. Een repository is een verspreidingsmedium; alles wat erin gaat, moet publiek kunnen worden.

Nuance. Een gecommit `.env` **zonder enig secret** (poorten, dienstnamen) is aanvaardbaar, en gebruikelijk met Compose (lab 09). Het probleem is de inhoud, niet het bestand.

Voorbeeld.

```
git log --all -p -- .env | grep -i password   # wat de geschiedenis bewaart
```

Vraag 13 — Lek in een ticket

Antwoord. (1) Het secret als **gecompromitteerd** beschouwen: op de vraag "wie heeft het gezien" bestaat geen betrouwbaar antwoord. (2) Het secret **roteren**: nieuw wachtwoord aan de databasekant, nieuwe waarde in de secretsbeheerder of `podman secret`, de gebruikers ervan opnieuw uitrollen. (3) Het oude **intrekken**, de toegangslogs van de database over die periode nakijken. (4) Het ticket opschonen, **daarna**, en de oorzaak aanpakken: waarom was een volledige `inspect` nodig, waarom bevatte die een secret (dus `-e` in plaats van een bestand). De opmerking verwijderen doet niets: ze is gelezen, per e-mail gemeld, misschien geïndexeerd. De snelheid hangt af van de architectuur: secret bij naam gerefereerd en bij het starten ingespoten → enkele minuten; secret met de hand gekopieerd in configuratiebestanden op servers → uren en vergetelheden.

Waarom. Het enige veilige antwoord op een lek is de gelekte waarde **waardeloos** maken.

Nuance. Dat is hét argument voor automatische rotatie: als een secret veranderen een dagelijkse, pijnloze handeling is, is een lek een klein incident.

Voorbeeld.

```
podman secret rm db_password && printf 'nieuw' | podman secret create db_password - && podman restart api db
```